

TAREA ACADÉMICA 6

ANÁLISIS POST-MORTEM Y AUDITORÍA DE CÓDIGO

“Vulnerabilidades, Deuda Técnica y Fallas Críticas Originadas por el Abuso de Aprendizaje JIT”

Auditor Principal: Verónica Godoy Serrano
Asignatura: Programación - Ciberseguridad y Calidad de Software
Fecha de Entrega: 30 de julio de 2026

Fundamentación: Conferencia del

Dr. Raj Reddy
“The Future of AI: Doomers vs. Abundance”
<https://www.youtube.com/watch?v=ydn0SMbyyQo>

Índice

1. Presentación del Caso de Auditoría	3
1.1. Contexto y Fundamentación	3
1.2. Expediente de Auditoría	3
2. Instrucciones para la Tarea	3
2.1. Diagnóstico Técnico de Vulnerabilidades	3
2.2. La Solución Metacognitiva (Aprender a Aprender)	3
2.3. Protocolo de Desarrollo Seguro con Asistentes de IA	3
3. Diagnóstico Técnico de Vulnerabilidades	4
3.1. Fallo 1: Inyección SQL Dinámica y Alucinada	4
3.1.1. Código Vulnerable Generado por IA	4
3.1.2. Análisis de Riesgo	4
3.1.3. ¿Por qué el Aprendizaje JIT impidió detectar el riesgo?	4
3.2. Fallo 2: Desbordamiento de Pila por Recursión Descontrolada	4
3.2.1. Código Vulnerable Generado por IA	4
3.2.2. Análisis de Riesgo	5
3.2.3. ¿Por qué el Aprendizaje JIT impidió detectar el riesgo?	5
3.3. Fallo 3: Condición de Carrera en Variables Globales Compartidas	5
3.3.1. Código Vulnerable Generado por IA	5
3.3.2. Análisis de Riesgo	5
3.3.3. ¿Por qué el Aprendizaje JIT impidió detectar el riesgo?	6
4. Solución Metacognitiva: Aprender a Aprender	7
4.1. Introducción	7
4.2. Solución para Fallo 1: Inyección SQL	7
4.2.1. Principios Fundamentales Aplicados	7
4.2.2. Código Refactorizado	7
4.2.3. Nota Analítica	7
4.3. Solución para Fallo 2: Desbordamiento de Pila	7
4.3.1. Principios Fundamentales Aplicados	7
4.3.2. Código Refactorizado	8
4.3.3. Nota Analítica	8
4.4. Solución para Fallo 3: Condición de Carrera	8
4.4.1. Principios Fundamentales Aplicados	8
4.4.2. Código Refactorizado	8
4.4.3. Nota Analítica	9
5. Protocolo de Desarrollo Seguro con Asistentes de IA	10
5.1. Política Institucional de 5 Puntos Obligatorios	10
5.2. Implementación del Protocolo	10
6. Conclusiones	11

1. Presentación del Caso de Auditoría

1.1. Contexto y Fundamentación

En la conferencia del Dr. Raj Reddy, se advierte que la llegada de agentes autónomos y la abundancia de código generado pueden tentar a los desarrolladores a renunciar al esfuerzo metacognitivo de comprender la arquitectura profunda del software. Cuando los equipos dependen exclusivamente de la adquisición de conocimiento Just-in-Time (JIT) (copiar parches de IA sin entender los fundamentos), los sistemas sufren de **incompetencia oculta**.

1.2. Expediente de Auditoría

Se presenta el expediente de auditoría de una aplicación financiera desplegada en la nube que colapsó debido a tres fallos graves:

1. **Inyección SQL Dinámica y Alucinada:** Un fragmento generado por IA utilizando concatenación de strings para resolver una consulta compleja “justo a tiempo”.
2. **Desbordamiento de Pila (Stack Overflow) por Recursión Descontrolada:** Generado al adaptar una solución JIT de ordenamiento en memoria sin verificar la profundidad de las llamadas para grandes volúmenes de datos.
3. **Condición de Carrera en Variables Globales Compartidas:** Producida al integrar librerías asíncronas sin comprender el bucle de eventos (Event Loop) ni el aislamiento de hilos.

2. Instrucciones para la Tarea

El estudiante actúa como **Auditor Principal de Ciberseguridad y Calidad de Software**, completando el siguiente informe:

2.1. Diagnóstico Técnico de Vulnerabilidades

Para cada uno de los 3 fallos, identificar por qué la técnica de *Aprendizaje Justo a Tiempo* impidió al equipo detectar el riesgo antes del despliegue en producción.

2.2. La Solución Metacognitiva (Aprender a Aprender)

Demostrar cómo el conocimiento de principios fundamentales (teoría de compilación, parsing seguro, estructuras de datos y concurrency models) habría permitido a un desarrollador entrenado en *Aprender a Aprender* identificar la falla de inmediato en el código sugerido por la IA.

2.3. Protocolo de Desarrollo Seguro con Asistentes de IA

Redactar una política institucional de 5 puntos obligatorios para la revisión de código (Code Review) cuando se utilicen herramientas de IA, reforzando la postura de Raj Reddy sobre la responsabilidad ineludible del supervisor humano en la era de la abundancia.

3. Diagnóstico Técnico de Vulnerabilidades

3.1. Fallo 1: Inyección SQL Dinámica y Alucinada

3.1.1. Código Vulnerable Generado por IA

```
1 def buscar_transacciones(filtros):
2     query = "SELECT * FROM transacciones WHERE 1=1"
3     if filtros.get("usuario"):
4         query += " AND usuario = '" + filtros["usuario"] + "'"
5     if filtros.get("fecha_inicio"):
6         query += " AND fecha >= '" + filtros["fecha_inicio"] + "'"
7     if filtros.get("monto_min"):
8         query += " AND monto >= " + str(filtros["monto_min"])
9     cursor.execute(query)
10    return cursor.fetchall()
```

Listing 1: Código vulnerable con concatenación de strings

3.1.2. Análisis de Riesgo

El código permite inyección SQL. Un atacante podría enviar:

```
1 ' OR '1'='1' --
```

Listing 2: Payload malicioso

Generando la consulta:

```
1 SELECT * FROM transacciones WHERE 1=1 AND usuario = '' OR '1'='1' --'
```

Listing 3: Consulta resultante peligrosa

Esta consulta devuelve todos los registros, exponiendo datos financieros sensibles.

3.1.3. ¿Por qué el Aprendizaje JIT impidió detectar el riesgo?

- El desarrollador copió el código de la IA sin comprender la **separación entre datos y comandos** en SQL.
- Desconocía la existencia de **consultas parametrizadas** como mecanismo de seguridad.
- No aplicó el principio de **mínimo privilegio** al construir la consulta.
- La IA generó código funcional pero inseguro, y el equipo lo aceptó sin cuestionarlo.

3.2. Fallo 2: Desbordamiento de Pila por Recursión Descontrolada

3.2.1. Código Vulnerable Generado por IA

```
1 def quicksort(lista):
2     if len(lista) <= 1:
3         return lista
4     pivote = lista[0]
5     izquierda = [x for x in lista[1:] if x <= pivote]
```

```
6 derecha = [x for x in lista[1:] if x > pivote]
7 return quicksort(izquierda) + [pivote] + quicksort(derecha)
```

Listing 4: Código vulnerable con recursión profunda

3.2.2. Análisis de Riesgo

Para listas de gran tamaño (ej. 1,000,000 registros), la recursión alcanza una profundidad igual al tamaño de la lista, provocando:

- Desbordamiento de la pila de llamadas (*Stack Overflow*).
- Caída del sistema en producción.
- Consumo excesivo de memoria.

3.2.3. ¿Por qué el Aprendizaje JIT impidió detectar el riesgo?

- El equipo adaptó la solución sin considerar la **complejidad espacial** del algoritmo.
- Desconocían el límite de recursión en Python (1000 llamadas por defecto).
- Ignoraron la posibilidad de implementar versiones iterativas o con optimización de cola.
- La IA generó código sintácticamente correcto pero algorítmicamente peligroso.

3.3. Fallo 3: Condición de Carrera en Variables Globales Compartidas

3.3.1. Código Vulnerable Generado por IA

```
1 saldo_global = 1000
2
3 async def procesar_transferencia(monto):
4     global saldo_global
5     if saldo_global >= monto:
6         await asyncio.sleep(0.1) # Simula operación asincrónica
7         saldo_global -= monto
8         return True
9     return False
```

Listing 5: Código vulnerable con condición de carrera

3.3.2. Análisis de Riesgo

Múltiples corrutinas ejecutando simultáneamente pueden:

- Leer el mismo saldo antes de que se actualice.
- Realizar transferencias que excedan el saldo disponible (sobregiro).
- Provocar pérdida de actualizaciones (*lost updates*).
- Generar inconsistencia en el estado financiero.

3.3.3. ¿Por qué el Aprendizaje JIT impidió detectar el riesgo?

- El equipo integró la librería asíncrona sin comprender el **bucle de eventos**.
- Desconocían la necesidad de **mecanismos de sincronización** (locks).
- Ignoraron el concepto de **atomicidad** en operaciones críticas.
- La IA generó código que parecía funcionar en pruebas unitarias pero fallaba en concurrencia real.

4. Solución Metacognitiva: Aprender a Aprender

4.1. Introducción

La solución metacognitiva implica comprender los principios fundamentales que subyacen a las tecnologías. Un desarrollador entrenado en *Aprender a Aprender* identifica fallos en código generado por IA al aplicar conocimiento teórico profundo, no solo sintaxis.

4.2. Solución para Fallo 1: Inyección SQL

4.2.1. Principios Fundamentales Aplicados

- **Teoría de Compilación:** El motor SQL compila la consulta en un AST. La concatenación permite que la entrada sea interpretada como código.
- **Parsing Seguro:** Uso de consultas parametrizadas para separar estructura de datos.

4.2.2. Código Refactorizado

```
1 def buscar_transacciones(filtros):
2     query = "SELECT * FROM transacciones WHERE 1=1"
3     condiciones = []
4     parametros = []
5
6     if filtros.get("usuario"):
7         condiciones.append("usuario = %s")
8         parametros.append(filtros["usuario"])
9     if filtros.get("fecha_inicio"):
10        condiciones.append("fecha >= %s")
11        parametros.append(filtros["fecha_inicio"])
12    if filtros.get("monto_min"):
13        condiciones.append("monto >= %s")
14        parametros.append(filtros["monto_min"])
15
16    if condiciones:
17        query += " AND " + " AND ".join(condiciones)
18
19    cursor.execute(query, parametros)
20    return cursor.fetchall()
```

Listing 6: Código seguro con consultas parametrizadas

4.2.3. Nota Analítica

El uso de placeholders (%s) garantiza que los datos sean tratados como datos, no como código ejecutable, eliminando la vulnerabilidad de inyección.

4.3. Solución para Fallo 2: Desbordamiento de Pila

4.3.1. Principios Fundamentales Aplicados

- **Estructuras de Datos:** Comprensión del límite de la pila de llamadas.

- **Análisis de Algoritmos:** Evaluación de complejidad espacial.
- **Técnicas de Optimización:** Transformación de recursión en iteración.

4.3.2. Código Refactorizado

```
1 def quicksort_iterativo(lista):
2     if len(lista) <= 1:
3         return lista
4
5     pila = [(0, len(lista) - 1)]
6     while pila:
7         inicio, fin = pila.pop()
8         if inicio >= fin:
9             continue
10
11         pivote = lista[fin]
12         i = inicio
13         for j in range(inicio, fin):
14             if lista[j] <= pivote:
15                 lista[i], lista[j] = lista[j], lista[i]
16                 i += 1
17         lista[i], lista[fin] = lista[fin], lista[i]
18
19         pila.append((inicio, i - 1))
20         pila.append((i + 1, fin))
21
22     return lista
```

Listing 7: Código seguro con implementación iterativa

4.3.3. Nota Analítica

La versión iterativa utiliza una pila explícita en el heap, evitando el desbordamiento de la pila de llamadas. La profundidad está controlada por el programador.

4.4. Solución para Fallo 3: Condición de Carrera

4.4.1. Principios Fundamentales Aplicados

- **Modelos de Concurrencia:** Comprensión de la diferencia entre concurrencia y paralelismo.
- **Sincronización:** Uso de cerrojos (locks) para garantizar atomicidad.

4.4.2. Código Refactorizado

```
1 import asyncio
2 import aiosqlite
3
4 class GestorSaldos:
5     def __init__(self, db_path):
6         self.db_path = db_path
7         self._lock = asyncio.Lock()
8
```



```
9     async def procesar_transferencia(self, cuenta_id, monto):
10         async with self._lock:
11             async with aiosqlite.connect(self.db_path) as db:
12                 cursor = await db.execute(
13                     "SELECT saldo FROM cuentas WHERE id = ?",
14                     (cuenta_id,)
15                 )
16                 row = await cursor.fetchone()
17                 if not row or row[0] < monto:
18                     return False
19
20                 await db.execute(
21                     "UPDATE cuentas SET saldo = saldo - ? WHERE id = ?",
22                     (monto, cuenta_id)
23                 )
24                 await db.commit()
25                 return True
```

Listing 8: Código seguro con sincronización

4.4.3. Nota Analítica

El cerrojo (`asyncio.Lock`) garantiza que solo una corrutina ejecute la operación crítica a la vez. La transacción SQL asegura atomicidad a nivel de base de datos.

5. Protocolo de Desarrollo Seguro con Asistentes de IA

5.1. Política Institucional de 5 Puntos Obligatorios

1. Revisión Manual de Código Generado por IA

- Todo código generado por asistentes de IA debe ser revisado por al menos un desarrollador senior.
- La revisión debe incluir análisis de seguridad, rendimiento y mantenibilidad.
- Se debe utilizar un checklist basado en OWASP Top 10.

2. Pruebas de Seguridad Automatizadas

- Cada fragmento de código debe superar pruebas estáticas (SAST) y dinámicas (DAST).
- Las pruebas deben incluir escenarios de inyección, desbordamiento y concurrencia.
- Mantener historial de pruebas para auditoría.

3. Capacitación Continua en Principios Fundamentales

- Capacitación periódica en fundamentos de programación, seguridad y arquitectura.
- Talleres prácticos sobre identificación de vulnerabilidades en código generado por IA.
- Fomentar la cultura de *Aprender a Aprender*.

4. Documentación y Trazabilidad del Código

- Todo código generado por IA debe estar etiquetado y documentado.
- Incluir justificación de por qué se aceptó el código.
- Documentación revisada por auditor de calidad.

5. Supervisión Humana y Responsabilidad Final

- El desarrollador que integra código generado por IA es responsable último.
- Establecer proceso de aprobación jerárquica para cambios críticos.
- Realizar análisis post-mortem en caso de fallo.

5.2. Implementación del Protocolo

- **Checklists de revisión** automatizados.
- **Integración continua** con análisis de seguridad.
- **Métricas de calidad** para evaluar código generado por IA.

6. Conclusiones

El análisis post-mortem de los tres fallos críticos evidencia los peligros de depender exclusivamente del aprendizaje Just-in-Time sin una comprensión profunda de los fundamentos de la programación.

La **solución metacognitiva**, basada en el principio de *Aprender a Aprender*, permite a los desarrolladores:

- Identificar vulnerabilidades en código generado por IA.
- Aplicar principios fundamentales para corregir fallos.
- Garantizar sistemas más seguros y robustos.

La implementación del **Protocolo de Desarrollo Seguro** con asistentes de IA —que incluye revisión manual, pruebas automatizadas, capacitación continua, documentación y supervisión humana— es esencial para mitigar los riesgos en la era de la abundancia de código generado.

Responsabilidad del Supervisor Humano

Como enfatiza el Dr. Raj Reddy, en la era de la abundancia, la responsabilidad ineludible del supervisor humano es garantizar que el código generado por IA sea seguro, confiable y ético. La tecnología es una herramienta, pero el juicio humano y el conocimiento profundo son insustituibles.

Referencias

- Reddy, R. (2026). *The Future of AI: Doomers vs. Abundance*. [Video]. Disponible en: <https://www.youtube.com/watch?v=ydn0SMbyyQo>
- OWASP. (2025). *OWASP Top 10 Web Application Security Risks*. Disponible en: <https://owasp.org/Top10/>
- Python Software Foundation. (2025). *sqlite3 — DB-API 2.0 interface for SQLite databases*. Disponible en: <https://docs.python.org/3/library/sqlite3.html>

Anexos

Anexo A: Código Refactorizado Completos

Los códigos refactorizados se encuentran en las secciones correspondientes:

- Sección 4.2: Solución Inyección SQL.
- Sección 4.3: Solución Desbordamiento de Pila.
- Sección 4.4: Solución Condición de Carrera.

Anexo B: Checklist de Revisión de Código

Checklist de Seguridad para Código Generado por IA
¿Se utilizan consultas parametrizadas para todas las interacciones con bases de datos?
¿Se evita la recursión sin límite de profundidad?
¿Se utilizan mecanismos de sincronización para operaciones concurrentes?
¿Se validan y sanitizan todas las entradas de usuario?
¿Se manejan adecuadamente las excepciones y errores?
¿Se ha realizado análisis estático y dinámico de seguridad?
¿La documentación explica el razonamiento detrás de las decisiones de implementación?